

Integration Manual

for S32K1_S32M24X ETH_43_ENET Driver

Document Number: IM2ETH_43_ENETASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	3
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Building the driver	6
3.1 Build Options	6
3.1.1 GCC Compiler/Assembler/Linker Options	6
3.1.1.1 GCC Compiler Options	6
3.1.1.2 GCC Assembler Options	8
3.1.1.3 GCC Linker Options	9
3.1.2 IAR Compiler/Assembler/Linker Options	9
3.1.2.1 IAR Compiler Options	9
3.1.2.2 IAR Assembler Options	10
3.1.2.3 IAR Linker Options	11
3.1.3 GHS Compiler/Assembler/Linker Options	11
3.1.3.1 GHS Compiler Options	11
3.1.3.2 GHS Assembler Options	13
3.1.3.3 GHS Linker Options	13
3.2 Files required for compilation	14
3.3 Setting up the plugins	16
4 Function calls to module	18
4.1 Function Calls during Start-up	18
4.2 Function Calls during Shutdown	18
4.3 Function Calls during Wake-up	18
5 Module requirements	19
5.1 Exclusive areas to be defined in BSW scheduler	19
5.2 Unavailable exclusive areas	19
5.3 Peripheral Hardware Requirements	19
5.4 ISR to configure within AutosarOS - dependencies	19
5.5 ISR Macro	20
5.5.1 Without an Operating System	20
5.5.1.1 Using Software Vector Mode	20
5.5.1.2 Using Hardware Vector Mode	20
5.5.2 With an Operating System	20
5.6 Other AUTOSAR modules - dependencies	20
5.7 Data Cache Restrictions	21
5.8 User Mode support	21
5.8.1 User Mode configuration in the module	21
5.8.2 User Mode configuration in AutosarOS	22
5.9 MultiPartition support	22
6 Main API Requirements	23
6.1 Main function calls within BSW scheduler	23
6.2 API Requirements	23
6.3 Calls to Notification Functions, Callbacks, Callouts	23
7 Memory allocation	24
7.1 Sections to be defined in Eth_MemMap.h	24



7.2 Linker command file	25
8 Integration Steps	26
9 External assumptions for driver	27

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This Integration Manual describes the integration requirements for NXP Semiconductors' AUTOSAR Ethernet Driver for S32K1.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176

All of the above microcontroller devices are collectively named as S32K1. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.

- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Default Error Tracer
ETH	Ethernet
ETHIF	Ethernet Interface
ETHTRCV	Ethernet Transceiver
ETHSWT	Ethernet Switch
MCU	Microcontroller Unit
MII	Media Independent Interface
N/A	Not Available
RMII	Reduced Media Independent Interface
RAM	Random Access Memory

- The term "Ethernet Controller" is related to the hardware module providing the Ethernet functionality.
- The term "Ethernet Driver" is related to the software handling the Ethernet Controller.
- The term "Application" is used for the software utilizing the Ethernet Driver.

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Building the driver

- [Build Options](#)
- [Files required for compilation](#)
- [Setting up the plugins](#)

This section describes the source files and various compilers, linker options used for building the driver. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- [GCC Compiler/Assembler/Linker Options](#)
- [IAR Compiler/Assembler/Linker Options](#)
- [GHS Compiler/Assembler/Linker Options](#)

The RTD driver files are compiled using:

- NXP GCC 10.2.0 20200723 (Build 1728 Revision g5963bc8)
- IAR ANSI C/C++ Compiler V8.40.3.228/W32 for ARM Functional Safety
- Green Hills Multi 7.1.6d / Compiler 2020.1.4

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D2M30I0R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D2 identifies S32K1 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)

Compiler Option	Description
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x or S32M24x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x or S32M24x devices)
-mfpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K11x devices)
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioned -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned

Compiler Option	Description
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↔ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↔ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32↔M24x devices)
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPO↔RT as a macro, with definition 1. Allows drivers to be configured in user mode.
-sysroot=	Specifies the path to the sysroot, for Cortex-M7 it is /arm-none-eabi/newlib
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-Xassembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mfpu=fpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x devices)
-mfpu=auto	Specifies the floating-point hardware available on the target (for S32K11x devices)

Assembler Option	Description
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x or S32M24x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x or S32M24x devices)
-mfpu=auto	Specifies the floating-point hardware available on the target (for S32K11x devices)
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library
-n	Turn off page alignment of sections, and disable linking against shared libraries
-sysroot=	Specifies the path to the sysroot, for Cortex-M7 it is /arm-none-eabi/newlib
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.2 IAR Compiler/Assembler/Linker Options

3.1.2.1 IAR Compiler Options

Compiler Option	Description
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)

Compiler Option	Description
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-cpu_mode=thumb	Generates code that executes in Thumb state
-endian=little	Generate code for a processor running in little-endian mode
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x or S32M24x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-e	Enables all IAR C language extensions
-Ohz	Optimize for size. The compiler will emit AEABI attributes indicating the requested optimization goal. This information can be used by the linker to select smaller or faster variants of DLIB library functions
-debug	Makes the compiler include debugging information in the object modules. Including debug information will make the object files larger
-no_clustering	Disables static clustering optimizations. Static and global variables defined within the same module will not be arranged so that variables that are accessed in the same function are close to each other
-no_mem_idioms	Makes the compiler not optimize certain memory access patterns
-no_explicit_zero_opt	Do not treat explicit initializations to zero of static variables as zero initializations
-require_prototypes	Force the compiler to verify that all functions have proper prototypes. Generates an error otherwise
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages
-diag_suppress=Pa050	Suppresses diagnostic message Pa050
-DIAR	Predefine IAR as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode.
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.2.2 IAR Assembler Options

Assembler Option	Description
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-cpu_mode thumb	Selects the thumb mode for the assembler directive CODE
-g	Disables the automatic search for system include files
-r	Generates debug information

3.1.2.3 IAR Linker Options

Linker Option	Description
-map filename	Produces a map file
-config linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x or S32M24x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-entry __start	Treats __start as a root symbol and start label
-enable_stack_usage	Enables stack usage analysis. If a linker map file is produced, a stack usage chapter is included in the map file
-skip_dynamic_initialization	Dynamic initialization (typically initialization of C++ objects with static storage duration) will not be performed automatically during application startup
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages

3.1.3 GHS Compiler/Assembler/Linker Options

3.1.3.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm4	Selects target processor: Arm Cortex M4 (for S32K14x or S32M24x devices)
-cpu=cortexm0plus	Selects target processor: Arm Cortex M0+ (for S32K11x devices)
-thumb	Selects generating code that executes in Thumb state

Compiler Option	Description
-fpu=vfpv4_d16	Specifies hardware floating-point using the v4 version of the VFP instruction set, with 16 double-precision floating-point registers (for S32K14x or S32M24x devices)
-fsingle	Use hardware single-precision, software double-precision FP instructions (for S32K14x or S32M24x devices)
-fsoft	Specifies software floating-point (SFP) mode. This setting causes your target to use integer registers to hold floating-point data and use library subroutine calls to emulate floating-point operations (for S32K11x devices)
-C99	Use (strict ISO) C99 standard (without extensions)
-ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
-gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
-unsigned_chars	Let the type char be unsigned, like unsigned char
-unsigned_fields	Bitfields declared with an integer type are unsigned
-no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
-no_exceptions	Disables C++ support for exception handling
-no_slash_comment	C++ style // comments are not accepted and generate errors
-prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
-incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)

Compiler Option	Description
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.3.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm4	Selects target processor: Arm Cortex M4 (for S32K14x or S32M24x devices)
-cpu=cortexm0plus	Selects target processor: Arm Cortex M0+ (for S32K11x devices)
-fpu=vfpv4_d16	Specifies hardware floating-point using the v4 version of the VFP instruction set, with 16 double-precision floating-point registers (for S32K14x devices)
-fsingle	Use hardware single-precision, software double-precision FP instructions (for S32K14x devices)
-fsoft	Specifies software floating-point (SFP) mode. This setting causes your target to use integer registers to hold floating-point data and use library subroutine calls to emulate floating-point operations (for S32K11x devices)
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.3.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for the modules in this library is provided in the src/libstartup directory

Linker Option	Description
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library (for S32K14x or S32M24x devices)
-lind_sf	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library (for S32K11x devices)
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-keep=C40_Ip_AccessCode	Avoid linker remove function C40_Ip_AccessCode from Fls module because it is not referenced explicitly
-nostartfiles	Controls the start files to be linked into the executable

3.2 Files required for compilation

This section describes the include files required to compile, assemble and link the AUTOSAR Ethernet Driver for S32K1 microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

Ethernet Driver Files:

- Eth_43_ENET_TS_T40D2M30I0R0\src\Eth_43_ENET.c
- Eth_43_ENET_TS_T40D2M30I0R0\src\Eth_43_ENET_Irq.c
- Eth_43_ENET_TS_T40D2M30I0R0\src\Eth_43_ENET_Ipw.c
- Eth_43_ENET_TS_T40D2M30I0R0\src\Eth_43_ENET_Ipw_Irq.c
- Eth_43_ENET_TS_T40D2M30I0R0\src\Enet_Ip.c
- Eth_43_ENET_TS_T40D2M30I0R0\src\Enet_Ip_Irq.c
- Eth_43_ENET_TS_T40D2M30I0R0\src\Enet_Ip_Hw_Access.c
- Eth_43_ENET_TS_T40D2M30I0R0\include\Eth_43_ENET.h
- Eth_43_ENET_TS_T40D2M30I0R0\include\Eth_43_ENET_Internal.h
- Eth_43_ENET_TS_T40D2M30I0R0\include\Eth_43_ENET_Ipw.h
- Eth_43_ENET_TS_T40D2M30I0R0\include\Enet_Ip.h
- Eth_43_ENET_TS_T40D2M30I0R0\include\Enet_Ip_Types.h
- Eth_43_ENET_TS_T40D2M30I0R0\include\Enet_Ip_Irq.h
- Eth_43_ENET_TS_T40D2M30I0R0\include\Enet_Ip_Hw_Access.h

Ethernet Driver Generated Files (must be generated by the user using a configuration tool):

- Enet_Ip_Cfg.c
- Eth_43_ENET_Cfg.h
- Eth_43_ENET_Ipw_Cfg.h
- Enet_Ip_Cfg.h

- Eth_43_ENET_[VariantName]_PBcfg.c
- Eth_43_ENET_Ipw_[VariantName]_PBcfg.c
- Enet_Ip_[VariantName]_PBcfg.c
- Eth_43_ENET_[VariantName]_PBcfg.h
- Eth_43_ENET_Ipw_[VariantName]_PBcfg.h
- Enet_Ip_[VariantName]_PBcfg.h
- Enet_Ip_Features.h
- Enet_Ip_CfgDefines.h

Note

As a deviation from the standard:

- Eth_43_ENET_[VariantName]_PBcfg.c, Eth_43_ENET_Ipw_[VariantName]_PBcfg.c, Enet_Ip_↔[VariantName]_PBcfg.c - These files will contain the definition for all parameters that are variant aware, independent of the configuration class that will be selected (PC, LT, PB)
- Enet_Ip_Cfg.c - This file will contain the definition for all configuration structures containing only variables that are not variant aware, configured and generated only once. This file alone does not contain the whole structure needed by Eth_Init function to configure the driver. Based on the number of variants configured in the EcuC, there can be more than one configuration structure for one module even for VariantPreCompile.

Base Files:

- BaseNXP_TS_T40D2M30I0R0\include\Eth_GeneralTypes.h
- BaseNXP_TS_T40D2M30I0R0\include\Mcal.h
- BaseNXP_TS_T40D2M30I0R0\include\Eth_43_ENET_MemMap.h
- BaseNXP_TS_T40D2M30I0R0\include\Platform_Types.h
- BaseNXP_TS_T40D2M30I0R0\include\Std_Types.h
- BaseNXP_TS_T40D2M30I0R0\include\OsIf.h
- BaseNXP_TS_T40D2M30I0R0\header\S32K148_ENET.h
- BaseNXP_TS_T40D2M30I0R0\header\S32K148_SIM.h
- BaseNXP_TS_T40D2M30I0R0\generate_PC\include\Soc_Ips.h
- BaseNXP_TS_T40D2M30I0R0\generate_PC\include\modules.h

DEM Files:

- Dem_TS_T40D2M30I0R0\include\Dem.h
- Dem_TS_T40D2M30I0R0\include\Dem_Types.h
- Dem_TS_T40D2M30I0R0\include\Dem_stub.h
- Dem_TS_T40D2M30I0R0\generate_PC\include\Dem_IntErrId.h
- Dem_TS_T40D2M30I0R0\src\Dem.c
- Dem_TS_T40D2M30I0R0\src\Dem_stub.c

DET Files:

- Det_TS_T40D2M30I0R0\include\Det_stub.h
- Det_TS_T40D2M30I0R0\include\Det.h
- Det_TS_T40D2M30I0R0\include\Det_LinkedList.h
- Det_TS_T40D2M30I0R0\src\Det_stub.c
- Det_TS_T40D2M30I0R0\src\Det.c

RTE Files:

- Rte_TS_T40D2M30I0R0\include\SchM_Eth_43_ENET.h
- Rte_TS_T40D2M30I0R0\src\SchM_Eth_43_ENET.c

EthIf Files:

- EthIf_TS_T40D2M30I0R0\include\EthIf.h
- EthIf_TS_T40D2M30I0R0\src\EthIf.c

EthTrcv Files:

- EthTrcv_TS_T40D2M30I0R0\include\EthTrcv.h
- EthTrcv_TS_T40D2M30I0R0\src\EthTrcv.c

EthSwt Files:

- EthSwt_TS_T40D2M30I0R0\include\EthSwt.h
- EthSwt_TS_T40D2M30I0R0\src\EthSwt.c

MKA Files:

- Mka_TS_T40D2M30I0R0\include\Mka.h

3.3 Setting up the plugins

The Ethernet Driver was designed to be configured by using the EB Tresos Studio (version 29.0.0 or later)

Location of various files inside the ETH module folder:

- VSMD (Vendor Specific Module Definition) file in EB Tresos Studio XDM format:
 - Eth_43_ENET_TS_T40D2M30I0R0\config\Eth_43_ENET.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - Eth_43_ENET_TS_T40D2M30I0R0\autosar\Eth_43_ENET_<subderivative_name>.epd
- Code Generation Templates for variant aware parameters:
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PB\src\Eth_43_ENET_PBcfg.c
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PB\src\Eth_43_ENET_Ipw_PBcfg.c
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PB\src\Enet_Ip_PBcfg.c
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PB\include\Eth_43_ENET_PBcfg.h

- Eth_43_ENET_TS_T40D2M30I0R0\generate_PB\include\Eth_43_ENET_Ipw_PBcfg.h
- Eth_43_ENET_TS_T40D2M30I0R0\generate_PB\include\Enet_Ip_PBcfg.h
- Code Generation Templates for parameters without variation points:
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PC\src\Enet_Ip_Cfg.c
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PC\include\Eth_43_ENET_Cfg.h
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PC\include\Eth_43_ENET_Ipw_Cfg.h
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PC\include\Enet_Ip_Cfg.h
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PC\include\Enet_Ip_CfgDefines.h
 - Eth_43_ENET_TS_T40D2M30I0R0\generate_PC\include\Enet_Ip_Features.h

Steps to generate the configuration:

1. Copy the following module folders into the Tresos plugins folder:
 - Eth_43_ENET_TS_T40D2M30I0R0
 - BaseNXP_TS_T40D2M30I0R0
 - Dem_TS_T40D2M30I0R0
 - Det_TS_T40D2M30I0R0
 - EcuC_TS_T40D2M30I0R0
 - Rte_TS_T40D2M30I0R0
 - Resource_TS_T40D2M30I0R0
 - Mcu_TS_T40D2M30I0R0
 - EthIf_TS_T40D2M30I0R0
 - EthTrev_TS_T40D2M30I0R0
 - EthSwt_TS_T40D2M30I0R0
 - Mka_TS_T40D2M30I0R0
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB Tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files

Chapter 4

Function calls to module

- [Function Calls during Start-up](#)
- [Function Calls during Shutdown](#)
- [Function Calls during Wake-up](#)

4.1 Function Calls during Start-up

\Note The Mcu initialization must always be removed for releases on VDK!

The Ethernet Driver shall be initialized through the *Eth_43*[infix]_Init_ API function during the start-up phase of EcuM (which means this module will be initialized by the BswM to allow for a short STARTUP phase). The Mcu and Port drivers shall be initialized beforehand (Port must be initialized before Mcu). The Ethernet Transceiver connected to the MAC has to be configured to the appropriate interface **prior** to the Ethernet Driver initialization. Configure fast slew-rate (and/or drive strength) for all Ethernet pins (otherwise packet loss may occur).

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- Unavailable exclusive areas
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- MultiPartition support

5.1 Exclusive areas to be defined in BSW scheduler

No exclusive areas have been identified.

5.2 Unavailable exclusive areas

None

5.3 Peripheral Hardware Requirements

None.

5.4 ISR to configure within AutosarOS - dependencies

The following ISRs are used by the Ethernet Driver when interrupts are switched on (the driver can also be run in polling mode):

ISR Name	NVIC Interrupt ID
ENET0_TIMESTAMP_IRQHandler	72
ENET0_RING0_TX_IRQHandler	73
ENET0_RING0_RX_IRQHandler	74
ENET0_ERR_IRQHandler	75

ISR Name	NVIC Interrupt ID
ENET0_WAKEUP_IRQHandler	77

5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.5.1 Without an Operating System

The macro `USING_OS_AUTOSAROS` must not be defined.

5.5.1.1 Using Software Vector Mode

The macro `USE_SW_VECTOR_MODE` must be defined and the ISR macro is defined as: `#define ISR(IsrName) void IsrName(void)`

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.5.1.2 Using Hardware Vector Mode

The macro `USE_SW_VECTOR_MODE` must not be defined and the ISR macro is defined as: `#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)`

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.5.2 With an Operating System

Please refer to your OS documentation for description of the ISR macro.

5.6 Other AUTOSAR modules - dependencies

- **Port:** Needed to configure the pins to be used by the Ethernet driver.
- **DET:** Needed for detecting and reporting development errors. The API function used is `Det_ReportError`. The detection can be configured using the `EthDevErrorDetect` configuration parameter
- **DEM:** Needed for detecting and reporting extended production errors.
- **RTE:** Needed for implementing data consistency through exclusive areas.
- **MCU:** Needed to configure the clocks to be used by the Ethernet driver.
- **EcuC:** Needed to retrieve information about post-build variants.
- **Base:** Needed for common files/definitions which are used by all RTD modules.
- **OS:** Needed to define a mapping between EcuC partitions and EcuC core ids when multicore support is enabled.
- **EthSwt_43_NETC:** Needed to configure switch port which are used for ethernet traffic.
- **EthIf:** The callbacks `EthIf_RxIndication` and `EthIf_TxConfirmation` are used to notify the upper layer about an ethernet frame transmission and reception. The callback `EthIf_CtrlModeIndication` is used to notify the upper layer about mode transitions in the controllers (i.e. ACTIVE -> DOWN or DOWN -> ACTIVE)
- **EthTrcv:** The callbacks `_EthTrcv_ReadMiiIndication` and `_EthTrcv_WriteMiiIndication` (or their vendor API infix variation) are used to notify the ethernet transceiver about a successful MDIO transfer.

- **EthSwt:** The callbacks *EthSwt_TxAdaptBufferLengthFunction* and *EthSwt_TxPrepareFrameFunction* are used to request the ethernet switch to do the preparations for a Tx buffer. The callbacks *EthSwt_TxProcessFrameFunction* and *EthSwt_TxFinishedIndicationFunction* are used to request the ethernet switch to process the ethernet frame before transmission. The callbacks *EthSwt_RxProcessFrameFunction* and *EthSwt_RxFinishedIndicationFunction* are used to notify the ethernet switch of a received ethernet frame.

5.7 Data Cache Restrictions

The descriptors are placed in a no-cacheable memory section delimited by *ETH_START_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE* and *ETH_STOP_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE*. As long as the MPU is properly configured to define the memory region containing the aforementioned section as NO-cacheable, there should be no data cache coherency problems.

The data buffers used in transfers can be internally allocated or externally. In both cases, they can be placed in cachable or NO-cacheable memory sections based on a configuration parameter:

1. **cache management enabled** *EthGeneral/EthGeneralVendorSpecific/EthEnableCacheManagement* node is checked, meaning that the driver ensures cache management for the data buffers. In this case, data buffers must be placed in cacheable memory section delimited by *ETH_START_SEC_VAR_CLEARED_UNSPECIFIED* and *ETH_STOP_SEC_VAR_CLEARED_UNSPECIFIED*.
2. **cache management disabled** *EthGeneral/EthGeneralVendorSpecific/EthEnableCacheManagement* node is not checked. In this case, data buffers must be placed in no cacheable memory section delimited by *ETH_START_SEC_VAR_CLEARED_UNSPECIFIED_NO_CACHEABLE* and *ETH_STOP_SEC_VAR_CLEARED_UNSPECIFIED_NO_CACHEABLE*.

Note

The proper infix must be used for the memory sections name.

5.8 User Mode support

- [User Mode configuration in the module](#)
- [User Mode configuration in AutosarOS](#)

5.8.1 User Mode configuration in the module

The Eth can be run in user mode if the following steps are performed:

- Enable **EthEnableUserModeSupport** from the configuration
- Call the following functions as trusted functions:

Function syntax	Description	Available via
void Enet_Ip_SetUserAccessAllowed(const ENET_Type *Base)	Set the UAA bit in REG_PROT to make the Instance accessible in user mode.	Enet_Ip_TrustedFunctions.h
void Enet_Ip_ClrUserAccessAllowed(const ENET_Type *Base)	Clears the UAA bit in REG_PROT to make the Instance unaccessible in user mode.	Enet_Ip_TrustedFunctions.h
void Eth_Ipw_RmiiClkOBE(void)	Output Buffer Enable for ENET RMII clock in internal loopback mode.	Eth_Ipw_TrustedFunctions.h

5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may have the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_IpTrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter [User Mode configuration in the module](#) for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

```
Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)
```

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

```
#define OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)
```

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.
- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will un-marshalling of the parameters to call `<Function_Name>()` of driver. The `<Function_Name>()` functions are already defined in driver and declared in `<IpName>_IpTrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

5.9 MultiPartition support

None.

Chapter 6

Main API Requirements

- [Main function calls within BSW scheduler](#)
- [API Requirements](#)
- [Calls to Notification Functions, Callbacks, Callouts](#)

6.1 Main function calls within BSW scheduler

The Ethernet Driver supports a main function that can be configured to be scheduled by the BSW Scheduler:

```
void Eth_[infix]_MainFunction(void);
```

This function checks for controller errors and lost frames and it is also used for polling state changes. Calls *Eth↔If_CtrlModeIndication* when the controller mode is changed.

6.2 API Requirements

None

6.3 Calls to Notification Functions, Callbacks, Callouts

The Ethernet Driver provides no configurable notification functions, callbacks, or callouts.

Chapter 7

Memory allocation

- [Sections to be defined in Eth_MemMap.h](#)
- [Linker command file](#)

7.1 Sections to be defined in Eth_MemMap.h

Section Name	Section Type	Description
ETH_START_SEC_CONFIG_DATA↔ _UNSPECIFIED	Configuration Data	Start of Memory Section for Config Data
ETH_STOP_SEC_CONFIG_DATA↔ UNSPECIFIED	Configuration Data	End of Memory Section for Config Data
ETH_START_SEC_CODE	Code	Start of Memory Section for Code
ETH_STOP_SEC_CODE	Code	End of Memory Section for Code
ETH_START_SEC_VAR_CLEARED↔ _UNSPECIFIED	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. These variables are never cleared and never initialized by start-up code
ETH_STOP_SEC_VAR_CLEARED↔ UNSPECIFIED	Variables	End of the above section
ETH_START_SEC_VAR_CLEARED↔ _UNSPECIFIED_NO_CACHEABLE	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. Normally, this section is used to store descriptors and data buffers. This section must also be cache inhibited
ETH_STOP_SEC_VAR_CLEARED↔ UNSPECIFIED_NO_CACHEABLE	Variables	End of the above section
ETH_START_SEC_VAR_CLEARED↔ _32	Variables	Used for variables which have to be aligned to 32 bits. For instance used for 32-bits variables or used for composite data types (arrays, structs) containing elements of maximum 32 bits. These variables are cleared to zero by start-up code.
ETH_STOP_SEC_VAR_CLEARED_32	Variables	End of the above section
ETH_START_SEC_CONST_32	Constants	Used for constants that have to be aligned to 32 bits
ETH_STOP_SEC_CONST_32	Constants	End of the above section

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>_MemMap.h".



Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section [Files Required for Compilation](#)
2. Allocate the proper memory sections in the driver's memory map header file ("`<Module>_MemMap.h`") and linker command file. For more details refer to section [Sections to be defined in `<Module>\_MemMap.h`](#)
3. Compile & build the module with all the dependent modules. For more details refer to section [Building the Driver](#)

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating the ETH driver into the application.

External Assumption Req ID	External Assumption Text
SWS_Eth_00026	Module: Header File: Imported TypeComStack_Types: ComStack_↵ Types.h: BufReq_ReturnTypeDem: Rte_Dem_Type.h: Dem_Event↵ IdTypeRte_Dem_Type.h: Dem_EventStatusTypeIcu: Icu.h: Icu_↵ ChannelTypeSpi: Spi.h: Spi_ChannelTypeSpi.h: Spi_DataBufferType↵ Spi.h: Spi_NumberOfDataTypeSpi.h: Spi_SequenceTypeSpi.h: Spi_↵ StatusTypeStd: Std_Types.h: Std_ReturnTypeStd_Types.h: Std_↵ VersionInfoType Note: Under control of BASE, DEM, RTE module
SWS_Eth_00159	Name: Eth_StateTypeKind: EnumerationRange: ETH_STATE_UNIN↵ IT: 0x00: Driver is not yet configuredETH_STATE_INIT: 0x01: Driver is configuredDescription: Status supervision used for Development Error Detection. The state shall be available for debugging.Available via: Eth_↵ GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00160	Name: Eth_FrameTypeKind: TypeDerived from: uint16Description: This type defines the Ethernet frame type used in the Ethernet frame header↵ Available via: Eth_GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00161	Name: Eth_DataTypeKind: TypeDerived from: Basetype↵ : Variationuint16: 8 or 16 bit CPUuint32: 32 bit CPUuint8: 8, 16 or 32 bit CPUDescription: This type defines the Ethernet data type used for data transmission. Its definition depends on the used CPU.Available via: Eth_GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00175	Name: Eth_BufIdxTypeKind: TypeDerived from: uint32Description: Eth- ernet buffer identifier type.Available via: Eth_GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00162	Name: Eth_RxStatusTypeKind: EnumerationRange: ETH_RECEIV↵ ED: 0x00: Ethernet frame has been received, no further frames availableE↵ TH_NOT_RECEIVED: 0x01: Ethernet frame has not been received, no further frames availableETH_RECEIVED_MORE_DATA_AVAILAB↵ LE: 0x02: Ethernet frame has been received, more frames are available↵ Description: Used as out parameter in Eth_Receive() indicates whether a frame has been received and if so, whether more frames are available or frames got lost.Available via: Eth_GeneralTypes.h Note: Under control of BASE module

External Assumption Req ID	External Assumption Text
SWS_Eth_00163	Name: Eth_FilterActionTypeKind: EnumerationRange: ETH_ADD_↵ TO_FILTER: 0x00: add the MAC address to the filter, meaning allow receptionETH_REMOVE_FROM_FILTER: 0x01: remove the MAC ad- dress from the filter, meaning reception is blocked in the lower layer↵ Description: The Enumeration Type Eth_FilterActionType describes the action to be taken for the MAC address given in *PhysAddrPtr.Available via: Eth_GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00177	Name: Eth_TimeStampQualTypeKind: EnumerationRange: ETH_VA↵ LID: 0: -ETH_INVALID: 1: -ETH_UNCERTAIN: 2: -Description: De- pending on the HW, quality information regarding the evaluated time stamp might be supported. If not supported, the value shall be always Valid. For Uncertain and Invalid values, the upper layer shall discard the time stamp.↵ Available via: Eth_GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00178	Name: Eth_TimeStampTypeKind: StructureElements: nanoseconds↵ Type: uint32Comment: Nanoseconds part of the timesecondsType: uint32↵ Comment: 32 bit LSB of the 48 bits Seconds part of the timesecondsHi↵ Type: uint16Comment: 16 bit MSB of the 48 bits Seconds part of the timeDescription: Variables of this type are used for expressing time stamps including relative time and absolute calendar time. The absolute time starts at 1970-01-01.: 0 to 281474976710655s == 3257812230d [0xFFFF FFFF F↵ FFF]: 0 to 999999999ns [0x3B9A C9FF] invalid value in nanoseconds: [0x3↵ B9A CA00] to [0x3FFF FFFF] Bit 30 and 31 reserved, default: 0Available via: Eth_GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00179	Name: Eth_TimeIntDiffTypeKind: StructureElements: diffType: Eth↵ _TimeStampTypeComment: time differencesignType: booleanComment: Positive (True) / negative (False) timeDescription: Variables of this type are used to express time differences.Available via: Eth_GeneralTypes.h Note: Under control of BASE module
SWS_Eth_00180	Name: Eth_RateRatioTypeKind: StructureElements: IngressTime↵ StampDeltaType: Eth_TimeIntDiffTypeComment: IngressTimeStamp↵ Sync2 - IngressTimeStampSync1OriginTimeStampDeltaType: Eth_↵ TimeIntDiffTypeComment: OriginTimeStampSync2[FUP2] - OriginTime↵ StampSync1[FUP1]Description: Variables of this type are used to express frequency ratios.Available via: Eth_GeneralTypes.h Note: Under control of BASE module

External Assumption Req ID	External Assumption Text
SWS_Eth_00120	API Function: Header File: Description Det_ReportError: Det.h: Service to report development errors. EthSwt_EthRxFinishedIndication: EthSwt_Eth.h: Indication for a finished receive process for a specific Ethernet frame, which results in providing the management information retrieved during EthSwt_EthRxProcessFrame(). EthSwt_EthRxProcessFrame: EthSwt_Eth.h: Function inspects the Ethernet frame passed by the data pointer for management information and stores it for later use in EthSwt_EthRxFinishedIndication(). EthSwt_EthTxAdaptBufferLength: EthSwt_Eth.h: Modifies the buffer length to be able to insert management information. EthSwt_EthTxFinishedIndication: EthSwt_Eth.h: Indication for a finished transmit process for a specific Ethernet frame. EthSwt_EthTxPrepareFrame: EthSwt_Eth.h: Prepares the Ethernet frame for common Ethernet communication (frame shall be handled by switch according to the common address resolution behavior) and stores the information for processing of EthSwt_EthTxFinishedIndication(). EthSwt_EthTxProcessFrame: EthSwt_Eth.h: Function inserts management information into the Ethernet frame. Icu_DisableNotification: Icu.h: This function disables the notification of a channel. Icu_EnableNotification: Icu.h: This function enables the notification on the given channel. Spi_GetStatus: Spi.h: Service returns the SPI Handler/Driver software module status. Spi_ReadIB: Spi.h: Service for reading synchronously one or more data from an IB SPI Handler/Driver Channel specified by parameter. Spi_SetupEB: Spi.h: Service to setup the buffers and the length of data for the EB SPI Handler/Driver Channel specified. Spi_SyncTransmit: Spi.h: Service to transmit data on the SPI bus. Spi_WriteIB: Spi.h: Service for writing one or more data to an IB SPI Handler/Driver Channel specified by parameter. Note: Under control EthSwt module
SWS_Eth_91001	Name: Eth_MacVlanTypeKind: StructureElements: MacAddrType: Array of uint8Size: 6Comment: Specifies the MAC address [0..255,0..255,0..255,0..255,0..255,0..255] VlanIdType: uint16Comment: Specifies the VLAN address 0..65535 SwitchPortType: uint32Comment: Specifies the ports of the switch as bit mask (0x00000001->Port0, 0x80000001->Port31+Port0) Description: This type is used to read out addresses from the address resolution logic (ARL) table of the switch. <pre>typedef struct { uint8 MacAddr[6U]; uint16 VlanId; uint32 SwitchPort; } Eth_MacVlanType;</pre> In case of Macaddr contains a Multicast Address MacVlanType.SwitchPort shall be handled as Bitmask, each bit represents a Switch Port, Bit 0 represents EthSwichtPortIdx = 0 , Bit 1 represents EthSwichtPortIdx = 1 and so on. In case of Macaddr contains not a Multicast Address MacVlanType.SwitchPort shall be handled as a value representing the EthSwitchPortIdx. Available via: Eth_GeneralTypes.h Note: Under BASE module, some HW don't support VLAN

External Assumption Req ID	External Assumption Text
SWS_Eth_91004	Name: Eth_TxErrorCounterValuesTypeKind: StructureElements: Tx↵DroppedNoErrorPktsType: uint32Comment: The number of outbound packets which were chosen to be discarded even though no errors had been detected to prevent their being transmitted. One possible reason for discarding such a packet could be to free up buffer space. Also described in IETF RFC1213 MIB ifOutDiscardsTxDroppedErrorPktsType: uint32Comment↵: transmitted because of errors. Also described in IETF RFC1213 MIB ifOutErrorsTxDeferredTransType: uint32Comment: A count of frames for which the first transmission attempt on a particular interface is delayed because the medium is busy. The count represented by an instance of this object does not include frames involved in collisions. Also described in IE↵TF RFC1643 MIB dot3StatsDeferredTransmissionsTxSingleCollisionType: uint32Comment: A count of successfully transmitted frames on a particular interface for which transmission is inhibited by exactly one collision. A frame that is counted by an instance of this object is also counted by the corresponding instance of either the TxUniCastPkts and TxNUcast↵Pkts and is not counted by the corresponding instance of the TxMultiple↵Collision object. Also described in IETF RFC1643 MIB dot3StatsSingle↵CollisionFramesTxMultipleCollisionType: uint32Comment: A count of successfully transmitted frames on a particular interface for which transmission is inhibited by more than one collision. A frame that is counted by an instance of this object is also counted by the corresponding instance of either the TxUniCastPkts and TxNUcastPkts and is not counted by the corresponding instance of the TxSingleCollision object. Also described in IETF RFC1643 MIB dot3StatsMultipleCollisionFrames.TxLateCollision↵Type: uint32Comment: The number of times that a collision is detected on a particular interface later than 512 bit-times into the transmission of a packet. Five hundred and twelve bit-times corresponds to 51.2 microseconds on a 10 Mbit/s system. A (late) collision included in a count represented by an instance of this object is also considered as a (generic) collision for purposes of other collision-related statistics. Also described in IETF RFC1643 MIB dot3StatsLateCollisionsTxExcessiveCollisonType: uint32Comment: A count of frames for which transmission on a particular interface fails due to excessive collisions. Also described in IETF RFC1643 MIB dot3Stats↵ExcessiveCollisionsDescription: Statistic counters for diagnostics.Available via: Eth_GeneralTypes.h Note: Under BASE module
SWS_Eth_91003	Name: Eth_TxStatsTypeKind: StructureElements: TxNumberOfOctets↵Type: uint32Comment: The total number of octets transmitted out of the interface, including framing characters. Also described in IETF RFC1213 MIB ifOutOctets.TxNUcastPktsType: uint32Comment: The total number of packets that higher-level protocols requested be transmitted to a non-unicast (i.e., a subnetwork-broadcast or subnetwork-multicast) address, including those that were discarded or not sent. Also described in IETF R↵FC1213 MIB ifOutNUcastPktsTxUniCastPktsType: uint32Comment: The total number of packets that higher-level protocols requested be transmitted to a subnetwork-unicast address, including those that were discarded or not sent. Also described in IETF RFC1213 MIB ifOutUcastPkts.Description: Statistic counter for diagnostics.Available via: Eth_GeneralTypes.h Note: Under BASE module

External Assumption Req ID	External Assumption Text
SWS_Eth_91002	Name: Eth_RxStatsTypeKind: StructureElements: RxStatsDropEvents← Type: uint32Comment: The total number of events in which packets were dropped by the probe due to lack of resources. Also described in I← ETF RFC 2819 MIB etherStatsDropEvents.RxStatsOctetsType: uint32← Comment: The total number of octets of data (including those in bad packets) received on the network (excluding framing bits but including F← CS octets). Also described in IETF RFC 2819 MIB etherStatsOctets.← RxStatsPktsType: uint32Comment: The total number of packets (includ- ing bad packets, broadcast packets, and multicast packets) received. Also described in IETF RFC 2819 MIB etherStatsPktsRxStatsBroadcastPkts← Type: uint32Comment: The total number of good packets received that were directed to the broadcast address. Also described in IETF RFC 2819 MIB etherStatsBroadcastPktsRxStatsMulticastPktsType: uint32Comment: The total number of good packets received that were directed to a multicast address. Also described in IETF RFC 2819 MIB etherStatsMulticast← Pkts.RxStatsCrcAlignErrorsType: uint32Comment: The total number of packets received that had a length of between 64 and 1518 octets that had either a bad Frame Check Sequence (FCS) with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error). Also described in IETF RFC 2819 MIB etherStatsCR← CAlignErrorsRxStatsUndersizePktsType: uint32Comment: The total num- ber of packets received that were less than 64 octets long (excluding framing bits, but including FCS octets) and were otherwise well formed. Also de- scribed in IETF RFC 2819 MIB etherStatsUndersizePkts.RxStatsOversize← PktsType: uint32Comment: The total number of packets received that were longer than 1518 octets (excluding framing bits, but including FCS octets) and were otherwise well formed. Also described in IETF RFC 2819 MIB etherStatsOversizePktsRxStatsFragmentsType: uint32Comment: The total number of packets received that were less than 64 octets in length (exclud- ing framing bits but including FCS octets) and had either a bad Frame Check Sequence (FCS) with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error). Also described in IETF RFC 2819 MIB etherStatsFragments.RxStatsJabbers← Type: uint32Comment: The total number of packets received that were longer than 1518 octets, and had either a bad Frame Check Sequence (F← CS) with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error). Also described in I← ETF RFC 2819 MIB etherStatsJabbers.RxStatsCollisionsType: uint32← Comment: The best estimate of the total number of collisions on this Ethernet segment. Also described in IETF RFC 2819 MIB etherStats← CollisionsRxStatsPkts64OctetsType: uint32Comment: The total number of packets (including bad packets) received that were 64 octets in length. Also described in IETF RFC 2819 MIB etherStatsPkts64OctetsRxStats← Pkts65to127OctetsType: uint32Comment: The total number of packets (in- cluding bad packets) received that were between 65 and 127 octets in length. Also described in IETF RFC 2819 MIB etherStatsPkts65to127OctetsRx← StatsPkts128to255OctetsType: uint32Comment: The total number of pack- ets (including bad packets) received that were between 128 and 255 octets in length. Also described in IETF RFC 2819 MIB etherStatsPkts128to255← OctetsRxStatsPkts256to511OctetsType: uint32Comment: The total num- ber of packets (including bad packets) received that were between 256 and 511 octets in length. Also described in IETF RFC 2819 MIB etherStats← Pkts256to511OctetsRxStatsPkts512to1023OctetsType: uint32Comment← : The total number of packets (including bad packets) received that were between 512 and 1023 octets in length. Also described in IETF RFC 2819 MIB etherStatsPkts512to1023OctetsRxStatsPkts1024to1518Octets← Type: uint32Comment: The total number of packets (including bad packets) received that were between 1024 and 1518 octets in length. Also described in
NXP Semiconductors	S32K144 S32M24X ETH 43 - ENET Driver

External Assumption Req ID	External Assumption Text
SWS_Eth_00260	The ECUC partitions referenced by EthCtrlEcucPartitionRef shall be a subset of the ECUC partitions referenced by EthEcucPartitionRef. Note: S32K1XX/SJA don't support multiple core
SWS_Eth_00261	EthCtrlConfig, EthTrcvConfig and EthSwtConfig (if existent in configuration) of one communication channel shall all reference the same ECUC partition. Note: S32K1XX/SJA don't support multiple core
SWS_Eth_91007	Name: Eth_CounterTypeKind: StructureElements: DropPktBufOverrunType: uint32Comment: dropped packets due to buffer overrunDropPktCrcType: uint32Comment: dropped packets due to CRC errorsUndersizePktType: uint32Comment: number of undersize packets which were less than 64 octets long (excluding framing bits, but including FCS octets) and were otherwise well formed. (see IETF RFC 1757)OversizePktType: uint32Comment: number of oversize packets which are longer than 1518 octets (excluding framing bits, but including FCS octets) and were otherwise well formed. (see IETF RFC 1757)AlgnmtErrType: uint32Comment: number of alignment errors, i.e. packets which are received and are not an integral number of octets in length and do not pass the CRC.SqeTestErrType: uint32Comment: SQE test error according to IETF RFC1643 dot3StatsSQETestErrorsDiscInbdPktType: uint32Comment: The number of inbound packets which were chosen to be discarded even though no errors had been detected to prevent their being deliverable to a higher-layer protocol. One possible reason for discarding such a packet could be to free up buffer space. (see IETF RFC 2233 ifInDiscards)ErrInbdPktType: uint32Comment: total number of erroneous inbound packetsDiscOtbdPktType: uint32Comment: The number of outbound packets which were chosen to be discarded even though no errors had been detected to prevent their being transmitted. One possible reason for discarding such a packet could be to free up buffer space. (see IETF RFC 2233 ifOutDiscards)ErrOtbdPktType: uint32Comment: total number of erroneous outbound packetsSnglCollPktType: uint32Comment: Single collision frames: A count of successfully transmitted frames on a particular interface for which transmission is inhibited by exactly one collision. (see IETF RFC1643 dot3StatsSingleCollisionFrames)MultCollPktType: uint32Comment: Multiple collision frames: A count of successfully transmitted frames on a particular interface for which transmission is inhibited by more than one collision. (see IETF RFC1643 dot3StatsMultipleCollisionFrames)DfrdPktType: uint32Comment: Number of deferred transmission: A count of frames for which the first transmission attempt on a particular interface is delayed because the medium is busy. (see IETF RFC1643 dot3StatsDeferredTransmissions)LatCollPktType: uint32Comment: Number of late collisions: The number of times that a collision is detected on a particular interface later than 512 bit-times into the transmission of a packet. (see IETF RFC1643 dot3StatsLateCollisions)HwDepCtr0Type: uint32Comment: hardware dependent counter valueHwDepCtr1Type: uint32Comment: hardware dependent counter valueHwDepCtr2Type: uint32Comment: hardware dependent counter valueHwDepCtr3Type: uint32Comment: hardware dependent counter valueDescription: Statistic counter for diagnostics.Available via: Eth_GeneralTypes.h Note: Under Base module

External Assumption Req ID	External Assumption Text
SWS_Eth_91008	Name: Eth_ModeType (obsolete)Kind: EnumerationRange: ETH_MODE↵ DE_DOWN: 0x00: disable the Ethernet Rx/Tx communication and set its corresponding hardware to a lowpower sleep mode and initiate a sleep process, if the Ethernet hardware provide such a feature. E.g. request a sleep on data line for OA TC10 compatible Ethernet hardwareETH_↵ MODE_ACTIVE: 0x01: enable the Ethernet Rx/Tx communication and set its corresponding hardware to an power on modeETH_MODE_AC↵ TIVE_WITH_WAKEUP_REQUEST: 0x02: enable the Ethernet Rx/Tx communication , set its corresponding Ethernet hardware to an power on mode and request an wake-up on the network, if the Ethernet hardware provide a wake-up feature. E.g. wake-up on data line for OA TC10 com- patible Ethernet hardwareDescription: This is an generic type and used in the layers of the Ethernet communication stack (e.g. EthIf, Eth, EthSwt, EthTrcv) to enable and disable, respectively, the Ethernet communca- tion channel and set the corresponding hardware (e.g. Ethernet controller, Eth- ernet Switch port, Ethernet transceiver) to an lowpower sleep and power on mode, respectively.: Tags: atp.Status=obsoleteAvailable via: Eth_↵ GeneralTypes.h
SWS_Eth_00284	The Mcu module (SPAL see SPAL General) shall configure register settings that are "shared" with other modules.
SWS_Eth_91011	Name: Eth_ModeType (draft)Kind: EnumerationRange: ETH_MODE↵ _DOWN: 0x00: disable the Ethernet Rx/Tx communication and set its corresponding hardware to a low-power sleep mode and initiate a sleep pro- cess, if the Ethernet hardware provides such a feature. E.g. request a sleep on data line for OA TC10 compatible Ethernet hardwareETH_M↵ ODE_ACTIVE: 0x01: enable the Ethernet Rx/Tx communication and set its corresponding hardware to a power-on modeETH_MODE_ACTIVE↵ _WITH_WAKEUP_REQUEST: 0x02: enable the Ethernet Rx/Tx com- munication, set its corresponding Ethernet hardware to a power-on mode and request an wake-up on the network, if the Ethernet hardware provides a wake-up feature. E.g. wake-up on data line for OA TC10 compatible Ethernet hardwareETH_MODE_ACTIVE_TX_OFFLINE: 0x03: disable the Tx communication path. Please note, this is only used in EthIf to sup- port silent communicaton (see COMM_SILENT_COMMUNICATION). In silent communication all transmission requests are rejectedDescription: This is an generic type and used in the layers of the Ethernet communication stack (e.g. EthIf, Eth, EthSwt, EthTrcv) to enable and disable, respectively, the Ethernet communication channel and set the corresponding hardware (e.g. Ethernet controller, Ethernet Switch port, Ethernet transceiver) to a low- power sleep and power on mode, respectively. The type also supports to transfer a wake-up request from the services layer (ComM) to the commu- nication drivers (EthTrcv). This could be used e.g. for Ethernet hardware that has the capability to wake-up and sleep on data line (see OA TC10): Tags: atp.Status=draftAvailable via: Eth_GeneralTypes.h
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: Rationale: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00093	The application shall define EcucCoreIDs in a compact/consecutive order, starting from zero.

External assumptions for driver

External Assumption Req ID	External Assumption Text
EA_RTD_00096	The application shall pass the correct initialization pointer, specific to the partition in which the driver is to be used.
EA_RTD_00106	Standalone IP configuration and HL configuration of the same driver shall be done in the same project
EA_RTD_00107	The integrator shall use the IP interface only for hardware resources that were configured for standalone IP usage. Note: The integrator shall not directly use the IP interface for hardware resources that were allocated to be used in HL context.
EA_RTD_00108	The integrator shall use the IP interface to build a CDD, therefore the BSWMD will not contain reference to the IP interface
EA_RTD_00113	When RTD drivers are integrated with AutosarOS and User mode support is enabled, the integrator shall assure that the definition and declaration of all RTD functions needed to be called as trusted functions follow the naming convention <code>Call<Function_Name>TRUSTED(parameter1,parameter2,...)</code> in Integration/User code. They need to be visible in Os.h for the driver to call them. They will call RTD <code><Function_Name>()</code> as trusted functions in OS specific manner.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.